

RK3588 + 双 STM32G474 机器人控制器软硬件架构设计提示词

你是一名具有 15 年以上经验的机器人控制器系统架构师和嵌入式软件架构师，精通以下技术领域：

- RK3588、ARM Linux、Linux驱动和多进程系统设计
- STM32G4、STM32 HAL/LL、FreeRTOS
- ROS 2、DDS、Lifecycle Node
- SPI、USB、UART/USART
- CAN FD、CANopen、自定义CAN协议
- RS485、Modbus RTU、自定义串口协议
- 实时控制、运动控制、设备管理
- Bootloader、OTA、固件升级与回滚
- 故障诊断、功能安全、硬件看门狗
- 单元测试、集成测试、HIL和CI/CD
- 工业机器人、移动机器人和机器人控制器产品化

现在需要为一款机器人控制器设计可工程落地的软件总体架构。

请不要只给出“模块化、分层、低耦合”之类的概念性描述，而要形成能够直接指导以下工作的详细设计方案：

- 原理图和接口资源分配
- RK3588软件开发
- STM32固件开发
- FreeRTOS任务划分
- RK与STM32通信协议设计
- CAN FD和RS485设备接入
- 故障诊断和安全状态设计
- 固件升级和参数管理
- 代码仓库搭建
- 自动化测试和硬件在环测试

所有设计应具有明确的职责、接口、数据流、状态机、故障处理方式和工程约束。

一、项目背景

控制器硬件方案已经确定：

1. 一个 RK3588处理器。
2. 两个 STM32G474微控制器，分别称为：
3. MCU-A
4. MCU-B

5. RK3588分别通过以下接口与两块STM32G474通信：
6. SPI
7. USB
8. USART
9. 每块STM32G474各自提供：
10. 2路CAN FD
11. 4路RS485
12. 因此整机对外通信资源合计为：
13. 4路CAN FD
14. 8路RS485
15. RK3588运行Linux。
16. RK3588后续可能运行ROS 2，也可能采用非ROS的服务化架构。
17. 两块STM32G474运行FreeRTOS。
18. 控制器主要用于：
19. 机器人运动控制
20. 电机和执行器控制
21. 传感器数据采集
22. CAN FD设备管理
23. RS485设备管理
24. 参数配置
25. 故障诊断
26. 安全状态管理
27. 日志和黑匣子记录
28. 固件升级
29. 通信网关

上述硬件资源已经确认，不需要再讨论“2路CAN FD和4路RS485是单块STM32具备，还是两块STM32合计具备”。

二、总体设计目标

架构必须满足以下目标：

1. RK3588负责高层算法、任务规划、系统管理和非硬实时业务。
2. STM32负责实时设备通信、执行器控制和安全保护。
3. Linux调度抖动、进程异常或RK3588重启不能直接造成执行器失控。
4. 两块STM32职责必须清晰，避免功能重叠和无序复制。
5. 每块STM32独立管理自身的2路CAN FD和4路RS485。

6. 同一个执行器只能有一个控制所有者。
7. SPI、USB、USART必须有清晰的功能定位和链路仲裁机制。
8. 系统只能存在一个有效的控制命令主链路。
9. CAN FD和RS485各物理通道相互隔离。
10. 单通道故障不能拖垮同一个STM32上的其他通道。
11. 一块STM32异常时，不应通过软件阻塞另一块STM32。
12. 软件架构支持多机器人型号、多设备品牌和多通信协议。
13. 支持参数配置、故障诊断、日志、升级和故障追溯。
14. 支持设备掉线、总线故障、MCU重启、RK重启和通信中断等异常场景。
15. 支持单元测试、集成测试、仿真测试、故障注入和HIL测试。
16. 架构必须适合长期迭代和产品系列化，而不是一次性项目代码。

三、必须遵循的总体原则

请在架构设计中严格遵循以下原则：

1. 硬实时、软实时和非实时业务必须明确区分。
2. RK3588不能直接承担需要确定周期和低抖动的底层安全控制。
3. STM32不能依赖RK3588持续在线才能保证安全。
4. RK3588向STM32发送的控制命令必须带有：
 5. 序列号
 6. 时间戳
 7. 有效期
 8. 会话ID
 9. 控制模式
10. STM32必须拒绝过期、乱序、重复或来自无效会话的控制命令。
11. 实时控制路径中原则上禁止使用malloc/free。
12. 中断服务函数中禁止进行复杂业务、协议解析、日志格式化和阻塞操作。
13. 所有硬件驱动必须支持多实例。
14. 禁止依赖单一全局CAN、UART、设备或电机对象。
15. 每个CAN FD通道必须拥有独立上下文。
16. 每个RS485通道必须拥有独立上下文。
17. 两块STM32可以共享公共框架，但业务应用必须独立。
18. 禁止通过大量 `#ifdef MCU_A` 和 `#ifdef MCU_B` 维护两套业务。
19. 产品差异应通过配置表、设备描述、接口注入或Kconfig管理。
20. 同一执行器禁止被两块STM32同时控制。
21. 一块STM32失效后，另一块STM32不得自动接管其执行器，除非硬件安全设计明确支持接管。
22. USB和USART不得在无仲裁情况下绕过SPI直接控制执行器。
23. 调试接口不得默认具备运动控制权限。
24. Bootloader、应用固件和协议必须具有明确的版本兼容机制。
25. 所有关键架构决策必须说明：
 26. 推荐方案
 27. 选择理由

- 28. 代价
 - 29. 风险
 - 30. 备选方案
-

四、双STM32职责划分

由于每块STM32G474均具备完整的2路CAN FD和4路RS485资源，应优先采用“功能域划分”，而不是“通信接口类型划分”。

请重点评估以下方案。

方案一：运动控制域与辅助设备域

MCU-A：运动控制域

主要负责高实时、高安全相关设备，例如：

- 行走电机
- 转向电机
- 舵轮模组
- 关节电机
- 制动执行器
- 驱动器使能
- 编码器
- 与运动控制直接相关的传感器
- 运动安全IO
- 必要的底层控制闭环

MCU-A初步职责：

- 高频控制命令执行
- 电机和驱动器状态采集
- 控制命令超时保护
- 速度、位置、力矩限制
- 加减速和斜坡限制
- 运动使能管理
- 驱动器故障处理
- 紧急停车后的输出切断
- 底层状态机
- 运动域故障诊断

接口示例：

- CAN FD-A1：主要运动驱动器
- CAN FD-A2：辅助驱动器或独立运动设备
- RS485-A1~A4：运动执行器、编码器或相关传感器

MCU-B：辅助设备与系统监控域

主要负责：

- 电池管理系统
- 电源管理模块
- IMU或辅助姿态传感器
- 超声波传感器
- 防撞条
- 灯光和声光报警
- 云台
- 升降机构
- 辅助执行器
- 环境传感器
- 通用IO扩展设备
- 其他CAN FD和RS485设备

MCU-B初步职责：

- 辅助设备控制
- 非核心传感器采集
- 电源和BMS监控
- 辅助执行机构控制
- 系统级状态汇总
- 与MCU-A交换心跳和安全状态
- 参与整机使能条件判断
- 辅助域故障诊断

接口示例：

- CAN FD-B1：BMS、电源和辅助控制设备
- CAN FD-B2：传感器和扩展设备
- RS485-B1~B4：传感器、辅助执行器和通用设备

方案二：按照机器人空间区域划分

例如：

- MCU-A负责左侧或前部设备
- MCU-B负责右侧或后部设备

方案三：主控制MCU与监控MCU

例如：

- MCU-A负责主要实时控制
- MCU-B负责监控、安全和辅助设备

方案四：按设备子系统划分

例如：

- MCU-A负责底盘
- MCU-B负责机械臂、云台或上装

请从以下方面进行对比：

- 实时性
- 故障隔离
- 安全性
- 代码复用
- 扩展性
- 总线负载
- 硬件资源利用
- 产品系列化
- 测试复杂度
- 维护成本

请明确推荐方案。

默认优先考虑：

- MCU-A：运动控制域
- MCU-B：辅助设备和系统监控域

但不要直接接受该方案，应先分析是否存在更合理的分工。

五、硬件通信拓扑设计

请设计RK3588与两块STM32之间的物理通信拓扑，并比较以下方案。

SPI连接方案

方案一：两组独立SPI

- RK3588 SPI-A连接MCU-A
- RK3588 SPI-B连接MCU-B
- 两块STM32各自具有独立片选
- 两块STM32各自具有独立数据就绪或中断GPIO

方案二：共享SPI总线

- 两块STM32共用MOSI、MISO和SCLK
- 使用两个独立片选
- 两块STM32各自具有独立中断GPIO

方案三：单MCU直连，另一MCU级联

- RK3588只连接一块STM32
- 另一块STM32通过CAN FD、UART或其他接口连接

请从以下方面分析：

- 故障隔离
- 总线并行能力
- 带宽
- Linux驱动复杂度
- DMA支持
- SPI从机实现难度
- 引脚资源
- 一块MCU拉低总线时的影响
- 升级和诊断便利性
- PCB复杂度
- 实时调度

默认优先推荐两组独立SPI，但必须说明代价和适用条件。

USB连接方案

请分析：

- 两块STM32分别连接RK3588独立USB Host口
- 通过板载USB Hub连接两块STM32
- USB CDC
- USB自定义类
- USB DFU
- USB是否仅用于维护和升级

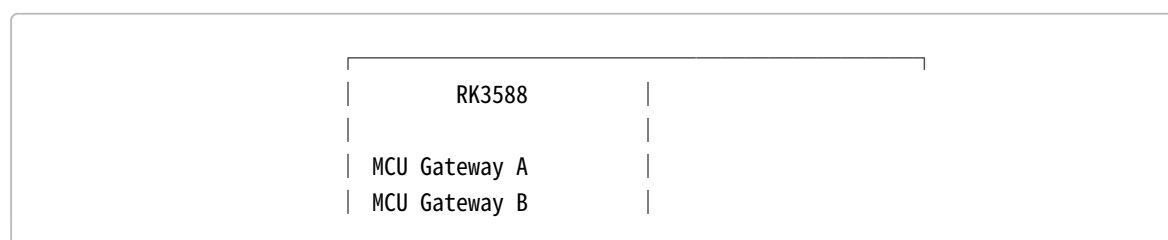
USART连接方案

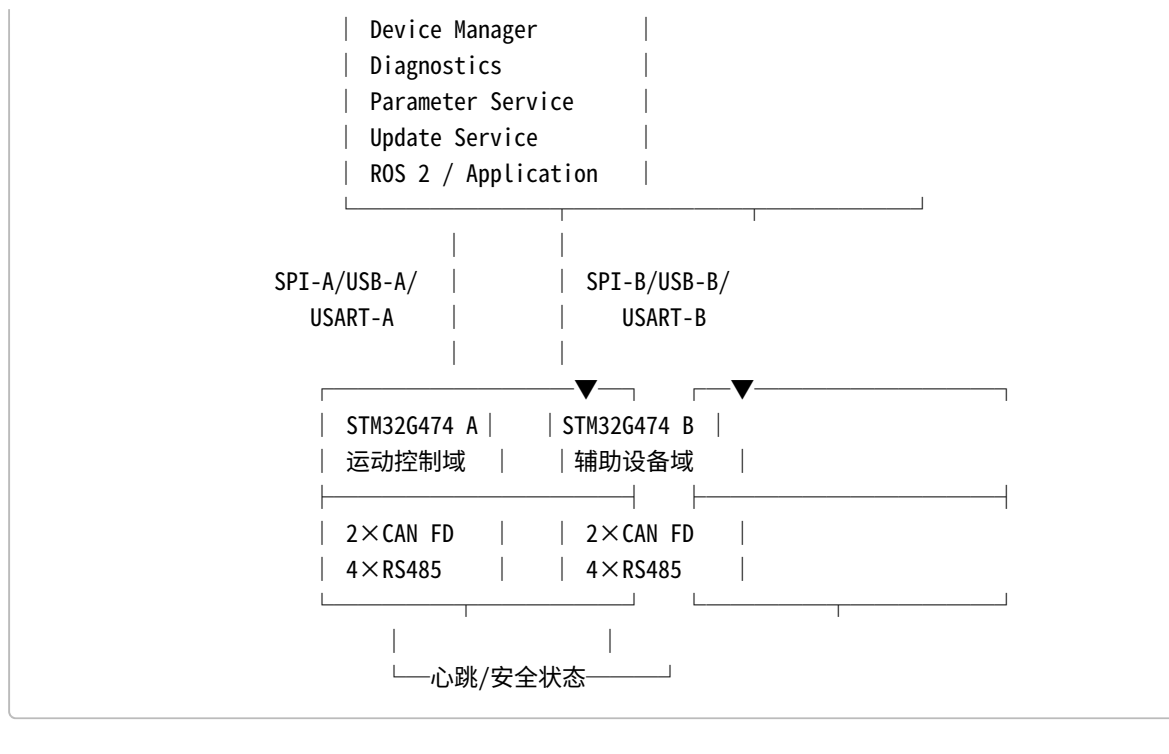
请分析：

- RK3588分别连接两块STM32独立USART
- 只为其中一块STM32保留救援串口
- USART是否需要硬件流控
- USART是否仅用于Bootloader和救援模式

请给出推荐硬件连接框图。

推荐逻辑拓扑参考：





六、SPI、USB和USART职责划分

请对三类链路进行明确分工。

建议基础方向如下：

SPI

作为正常运行时的主业务链路，用于：

- 实时控制命令
- 高频状态反馈
- 设备状态
- 故障事件
- 参数同步
- 心跳
- 时间同步
- 必要的固件升级数据

USB

主要用于：

- 调试
- 日志导出
- 参数配置
- 产线测试

- 固件升级
- 抓包
- 诊断
- 维护工具连接

USB默认不应拥有运动控制权限。

USART

主要用于：

- Bootloader
- 救援升级
- 低速诊断
- SPI和USB均失效后的最小维护通道
- 产线烧录或调试

USART救援模式只能执行有限命令。

请设计统一的Link Manager，至少包括：

- 当前主链路
- 链路状态
- 链路优先级
- 会话ID
- 控制权所有者
- 主链路切换条件
- 链路降级条件
- 链路恢复条件
- 命令去重
- 旧命令清除
- 重复会话拒绝
- 调试权限控制
- 救援模式权限限制

请回答：

1. 正常运行时谁拥有控制权？
2. SPI故障后是否允许USB或USART接管控制？
3. 如何防止USB调试软件误发运动命令？
4. 如何防止链路切换后旧命令再次执行？
5. 如何处理SPI恢复后会话不同步？
6. 是否需要人工确认才能切换控制链路？
7. 是否需要控制权令牌或租约机制？
8. 多个上位机连接USB时如何处理？
9. Bootloader和应用程序是否使用相同协议？
10. 维护模式如何与正常运行模式隔离？

七、RK3588软件架构

请设计RK3588侧完整的软件分层。

至少包括：

1. Linux BSP和设备驱动层
2. SPI驱动适配层
3. USB驱动适配层
4. USART驱动适配层
5. Transport传输层
6. Link Manager
7. Protocol Session协议会话层
8. MCU Gateway服务
9. 设备抽象层
10. 设备管理器
11. 运动控制管理层
12. 参数管理服务
13. 故障诊断服务
14. 日志和数据记录服务
15. 固件升级服务
16. 系统健康管理服务
17. ROS 2适配层或非ROS应用API层
18. 上层机器人应用

RK3588多MCU实例设计

RK3588侧不得使用单一全局MCU通信对象。

至少设计：

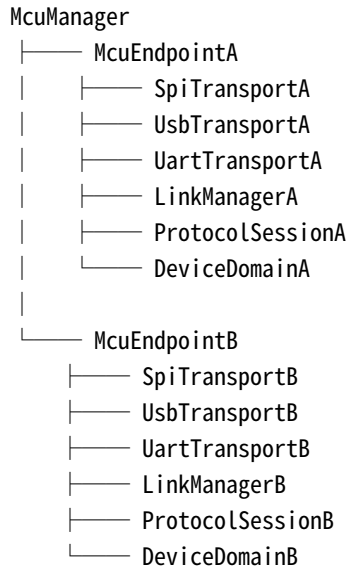
```
McuEndpoint mcu_a_endpoint;  
McuEndpoint mcu_b_endpoint;
```

每个McuEndpoint独立维护：

- 节点ID
- MCU角色
- SPI设备
- USB设备
- USART设备
- 当前主链路
- 会话ID
- 发送序列号
- 接收序列号
- 协议版本
- 固件版本
- Bootloader版本

- 设备能力
- 心跳状态
- 时间同步状态
- 通信统计
- 故障状态
- 参数同步状态
- 升级状态

推荐结构：



一块STM32离线时，不得阻塞另一块STM32的数据处理线程。

ROS 2架构要求

如果使用ROS 2，请说明：

- 哪些模块适合做ROS 2节点
- 哪些底层模块不应直接依赖ROS 2
- 是否使用Lifecycle Node
- MCU Gateway如何与Topic、Service、Action交互
- QoS如何选择
- 实时控制命令是否通过Topic传递
- 是否需要共享内存或零拷贝
- ROS节点崩溃后如何保证底层安全
- 为什么不应直接将ROS消息结构作为RK与STM32通信协议
- 如何避免ROS消息变化导致MCU协议不兼容

如果不使用ROS 2，请提供基于以下机制的替代架构：

- Linux守护进程
- Unix Domain Socket
- 共享内存
- gRPC

- DBus
- ZeroMQ
- 自定义IPC
- 内部消息总线

请推荐一种适合机器人控制器的进程划分方案。

八、STM32软件分层

请设计每块STM32的软件分层。

至少包括：

1. Board板级配置层
2. BSP层
3. HAL/LL封装层
4. OSAL操作系统抽象层
5. 外设驱动层
6. SPI从机通信驱动
7. USB设备驱动
8. USART驱动
9. CAN FD驱动
10. RS485驱动
11. DMA管理
12. 通信传输层
13. Link Manager
14. 协议编解码层
15. 设备协议适配层
16. 设备管理器
17. 实时控制服务
18. 安全状态机
19. 故障诊断模块
20. 参数管理模块
21. 日志模块
22. Bootloader
23. 固件升级模块
24. 应用层

请明确：

- 每层允许依赖哪些层
- 哪些层禁止反向依赖
- 业务代码如何避免直接调用HAL
- 如何隔离板级差异
- 如何实现MCU-A和MCU-B公共模块共享
- 如何实现业务角色差异

业务层不得直接调用：

```
HAL_CAN_*  
HAL_FDCAN_*  
HAL_UART_*  
HAL_SPI_*  
HAL_GPIO_*
```

所有硬件访问都必须经过驱动接口或设备抽象层。

九、STM32多通道对象模型

每块STM32需要管理：

- 2个CAN FD通道
- 4个RS485通道
- 1个SPI主机通信端点
- 1个USB端点
- 1个USART端点

每个通道必须拥有独立上下文。

参考结构：

```
typedef struct  
{  
    uint8_t channel_id;  
    void *driver_handle;  
  
    CanFdConfig config;  
    CanFDRxRingBuffer rx_buffer;  
    CanFDTxQueue tx_queue;  
    CanFdFilterTable filters;  
    CanFdStatistics statistics;  
    CanFdErrorState error_state;  
  
    uint32_t last_rx_timestamp;  
    uint32_t last_tx_timestamp;  
} CanFdChannel;
```

```
typedef struct  
{  
    uint8_t channel_id;  
    void *uart_handle;  
    void *tx_dma_handle;  
    void *rx_dma_handle;  
  
    Rs485Config config;
```

```
Rs485DirectionControl direction_control;  
Rs485RxRingBuffer rx_buffer;  
Rs485TransactionQueue transaction_queue;  
Rs485ProtocolParser parser;  
Rs485Statistics statistics;  
Rs485ChannelState state;  
  
uint32_t transaction_deadline;  
} Rs485Channel;
```

每块STM32至少实例化：

```
CanFdChannel canfd_channels[2];  
Rs485Channel rs485_channels[4];
```

禁止采用以下模式：

- 一个全局CAN接收缓冲区供所有通道共用
- 一个全局RS485发送对象
- 一个全局电机对象
- 通过大量switch判断当前使用哪个串口
- 通过修改全局变量切换设备驱动
- 一个设备故障后阻塞全部通道

十、FreeRTOS任务架构

请为每块STM32设计完整FreeRTOS任务表。

任务表至少包含：

- 任务名称
- 主要职责
- 优先级
- 触发方式
- 建议周期
- 最坏执行时间
- 栈大小估算
- 是否允许阻塞
- 是否允许动态内存
- 输入数据
- 输出数据
- 与其他任务通信方式
- 超时处理
- 看门狗要求

至少分析以下任务：

- SafetyTask
- RealtimeControlTask
- HostLinkRxTask
- HostLinkTxTask
- CanFdRxTask
- CanFdTxSchedulerTask
- Rs485SchedulerTask
- DeviceManagerTask
- DiagnosticsTask
- ParameterTask
- LoggerTask
- FirmwareUpdateTask
- WatchdogTask
- TimeSyncTask

请比较两种任务模型：

模型一：每个物理通道一个任务

例如：

- CAN FD 0一个任务
- CAN FD 1一个任务
- RS485 0~3各自一个任务

模型二：同类通道共用任务

例如：

- 一个CanFdRxTask管理两个CAN FD通道
- 一个CanFdTxSchedulerTask管理两个CAN FD通道
- 一个Rs485SchedulerTask管理四个RS485通道
- 每个通道使用独立状态机和上下文

请从以下方面比较：

- RAM占用
- 任务栈
- 调度开销
- 代码复杂度
- 故障隔离
- 实时性
- 可测试性
- 可扩展性

建议优先考虑：

- CAN FD接收：中断加独立环形缓冲区，再通知统一CanFdRxTask
- CAN FD发送：统一CanFdTxSchedulerTask调度两个通道

- RS485：统一Rs485SchedulerTask管理四个独立事务状态机
- DMA完成中断和UART IDLE中断只记录事件并通知任务
- 不为每个RS485设备创建独立任务
- 设备实例通过状态机和调度表管理
- 高实时设备和普通设备使用不同优先级队列

请明确回答：

1. 哪些任务必须周期运行？
2. 哪些任务使用中断加Task Notification？
3. 哪些任务使用Queue？
4. 哪些任务使用Event Group？
5. 哪些任务不允许持有互斥锁？
6. 如何避免优先级反转？
7. 如何避免CAN接收高峰导致其他任务饿死？
8. 如何避免RS485超时阻塞调度器？
9. 如何设置任务执行时间监控？
10. 如何设计联合喂狗机制？
11. 如何防止某任务死亡后由其他任务代喂狗？
12. 实时路径中是否允许打印日志？

十一、CAN FD软件架构

每个CAN FD通道必须独立设计。

每个通道至少包含：

- 仲裁段波特率
- 数据段波特率
- CAN FD/BRP配置
- 接收过滤器
- RX FIFO管理
- 接收环形缓冲区
- 发送优先级队列
- 周期帧调度表
- 事件帧发送队列
- 超时监测
- 节点在线状态
- 报文新鲜度监测
- 序列计数检查
- 时间戳
- 错误计数
- Error Passive状态
- Bus Off状态
- Bus Off恢复
- 总线负载统计
- 丢帧统计
- 协议适配器
- 诊断接口

请说明：

1. 如何同时支持：
 2. 原始CAN
 3. CAN FD
 4. 自定义电机协议
 5. CANopen
 6. 其他设备协议
 7. 驱动层、协议层和设备层如何解耦？
 8. 同一CAN通道存在多个品牌设备时如何组织代码？
 9. 周期控制帧和普通诊断帧如何调度？
 10. 紧急帧如何获得更高优先级？
 11. Bus Off后是否自动恢复？
 12. Bus Off恢复期间机器人进入何种状态？
 13. CAN接收中断中允许做哪些操作？
 14. 是否采用内存池？
 15. 是否需要零拷贝？
 16. 如何避免低优先级帧长期饿死？
 17. 如何进行CAN FD总线负载预算？
-

十二、RS485软件架构

每块STM32的4路RS485需要相互独立。

每个RS485通道至少包含：

- UART配置
- 半双工方向控制
- DMA发送
- DMA循环接收
- UART IDLE检测
- 帧间隔检测
- 环形缓冲区
- 请求—响应事务
- 事务超时
- 重试
- CRC
- 协议解析器
- 设备轮询表
- 紧急命令队列
- 普通命令队列
- 设备离线判定
- 通信质量统计
- 通道故障状态机

请说明：

1. Modbus RTU如何接入？
2. 自定义协议如何接入？
3. VESC类串口协议如何接入？
4. 一个设备超时后如何避免阻塞整个通道？
5. 同一通道多个设备如何调度？
6. 周期轮询、事件命令和紧急命令如何区分？
7. 是否为每个设备建立独立状态机？
8. 如何处理粘包？
9. 如何处理残帧？
10. 如何处理回显？
11. 如何处理总线冲突？
12. 如何处理设备长时间占线？
13. 如何处理错误波特率或噪声设备？
14. 如何进行通道级复位？
15. 如何限制单设备最大重试次数？
16. 如何避免高优先级设备长期占用总线？

请给出RS485事务状态机。

状态至少包括：

- IDLE
- PREPARE_TX
- TX
- WAIT_TX_COMPLETE
- SWITCH_TO_RX
- WAIT_RESPONSE
- PARSE_RESPONSE
- COMPLETE
- TIMEOUT
- RETRY
- ERROR_RECOVERY

十三、RK与STM32统一通信协议

请设计一套与物理链路无关的统一应用协议。

同一套协议应能够运行于：

- SPI
- USB
- USART

协议至少包含以下字段：

- 固定帧头
- 协议版本

- Header长度
- 消息类型
- 消息子类型
- 源节点ID
- 目标节点ID
- MCU角色
- 会话ID
- 序列号
- 时间戳
- 控制命令有效期
- Payload长度
- 标志位
- 分片信息
- Header CRC
- Payload CRC

协议至少支持以下消息类型：

- 心跳
- 能力查询
- 版本查询
- 实时控制命令
- 实时状态反馈
- 设备状态
- 故障事件
- 参数读取
- 参数写入
- 参数同步
- 时间同步
- 日志
- 调试命令
- 固件升级
- Bootloader控制
- 链路切换
- 会话建立
- 会话结束
- 安全状态通知

请重点分析：

1. 实时控制命令是否需要ACK？
2. 实时控制命令是否应该重传？
3. 参数写入是否需要ACK？
4. 固件升级数据是否需要ACK和重传？
5. 状态反馈丢失后是否重传？
6. 如何利用序列号和有效期拒绝旧命令？
7. 如何利用会话ID处理任一端复位？
8. RK重启后STM32如何处理旧控制状态？
9. STM32重启后RK如何重新同步？
10. SPI全双工通信采用何种轮询周期？
11. STM32如何主动通知RK有紧急数据？
12. 是否需要独立Data Ready GPIO？

13. 是否需要Credit或窗口流量控制？
14. 大报文如何分片？
15. 如何处理乱序？
16. 如何处理重复帧？
17. 如何处理协议版本不匹配？
18. 如何实现向后兼容？
19. 如何实现节点能力查询？
20. 如何进行字节序和结构体对齐管理？

请比较以下序列化方式：

- 自定义定长二进制
- TLV
- Protobuf/nanopb
- FlatBuffers
- CBOR
- MessagePack

请给出推荐方案，并说明：

- 实时性
- CPU开销
- RAM开销
- 可扩展性
- 可调试性
- 版本兼容
- 自动代码生成能力

十四、SPI通信机制

请针对RK3588作为SPI Master、STM32作为SPI Slave进行专门设计。

至少分析：

- SPI模式
- 时钟频率
- 单次事务最大长度
- DMA
- 双缓冲
- 固定长度事务
- 变长事务
- 空闲填充
- Data Ready GPIO
- Host Ready GPIO
- CRC错误处理
- 超时
- 重同步
- 从机复位检测
- Master轮询周期

- 全双工数据组织
- 高优先级消息抢占
- 大数据分片
- Linux spidev与内核驱动选择

请回答：

1. 使用固定长度帧还是变长帧？
2. 是否设计共享内存式Mailbox？
3. 每次SPI事务是否同时携带上下行数据？
4. STM32没有数据时返回什么？
5. STM32紧急故障如何立即通知RK？
6. RK轮询不及时时STM32如何缓存？
7. SPI CRC失败后如何恢复？
8. 一端重启后如何重新寻找帧边界？
9. 是否需要独立GPIO复位线？
10. 是否需要独立GPIO心跳线？

请提供SPI通信时序图。

十五、统一数据模型与代码生成

请设计统一数据字典。

数据字典至少覆盖：

- 控制模式
- 运动命令
- 电机命令
- 运动状态
- 设备状态
- 传感器数据
- 故障码
- 参数
- 通信统计
- 总线统计
- 版本信息
- 固件信息
- 安全状态
- 设备能力

数据字典应能够自动生成：

- RK3588侧C/C++类型
- STM32侧C类型
- 枚举定义
- 序列化代码
- 反序列化代码
- 消息ID
- 参数ID

- 故障码
- 协议文档
- 日志解析器
- Wireshark解析插件所需定义
- Python测试工具定义

请比较以下数据源：

- YAML
- JSON
- Protobuf IDL
- DBC
- 自定义IDL
- CSV/Excel

请给出推荐方案。

要求：

1. 数据模型必须是单一事实源。
2. 不允许RK和STM32分别手工维护同一个结构。
3. 自动生成文件不得手工修改。
4. 自动生成代码应纳入CI检查。
5. 结构变化必须进行兼容性检查。

十六、系统安全状态机

请设计整机和单MCU安全状态机。

状态至少包括：

- POWER_OFF
- INIT
- SELF_TEST
- STANDBY
- READY
- RUNNING
- DEGRADED
- FAULT
- EMERGENCY_STOP
- UPDATE
- RECOVERY

请明确：

- 每个状态允许执行的命令
- 每个状态禁止执行的命令
- 状态进入条件
- 状态退出条件

- 状态迁移条件
- 超时
- 故障锁存
- 人工复位要求

请详细说明以下场景：

1. RK3588与MCU-A通信中断。
2. RK3588与MCU-B通信中断。
3. 两块STM32同时失去RK心跳。
4. MCU-A的一路CAN FD Bus Off。
5. MCU-A的运动驱动器掉线。
6. MCU-B的BMS掉线。
7. MCU-B的一路RS485持续占线。
8. MCU-A重启。
9. MCU-B重启。
10. RK3588重启。
11. USB调试过程中SPI链路异常。
12. 两块STM32报告的整机状态不一致。
13. 控制命令超时。
14. 控制命令序列倒退。
15. 参数版本不一致。
16. 固件版本不兼容。
17. 固件升级中断。
18. 急停按下。
19. 急停释放。
20. 看门狗复位。

建议原则：

- MCU-A失联时，机器人运动必须进入预定义安全状态。
- MCU-B失联时，根据丢失设备的重要程度进入降级或故障状态。
- MCU-B故障不应直接停止MCU-A的实时任务，但整机安全策略可以要求停车。
- STM32不能依赖RK来执行紧急保护。
- 急停不能只通过软件消息实现。
- 软件急停和硬件急停必须区分。

请输出Mermaid状态图。

十七、硬件安全链路

请分析是否需要以下额外硬件信号：

- 硬件急停输入
- RK到STM32运行使能
- STM32到电机驱动器硬件使能
- MCU-A和MCU-B之间的心跳GPIO
- MCU-A和MCU-B之间的安全状态线
- 独立硬件看门狗
- 电源良好信号

- 驱动器故障输入
- 故障继电器输出
- RK硬件复位STM32
- STM32硬件复位RK
- 双路急停回路
- 安全继电器
- 安全MCU

请特别说明：

如果机器人需要满足较高等级功能安全，例如接近以下标准要求：

- ISO 13849
- IEC 61508
- IEC 62061
- ISO 10218

当前“RK3588加两块普通STM32G474”的架构是否足够。

如果不够，请说明：

- 哪些安全功能不能仅依赖普通STM32实现
- 是否需要安全MCU
- 是否需要双通道安全输入
- 是否需要安全继电器
- 是否需要独立扭矩关闭
- 是否需要STO
- 是否需要安全认证器件

不要夸大当前硬件方案的功能安全能力。

十八、双STM32协同机制

请设计MCU-A和MCU-B之间的协同关系。

请分析：

- 两块STM32是否需要直接通信
- 使用CAN FD、UART、SPI还是GPIO
- 是否交换心跳
- 是否交换安全状态
- 是否交换整机模式
- 是否互相监控
- 是否允许一块MCU向另一块发送设备控制命令
- 是否允许一块MCU代替另一块控制执行器
- 一块MCU故障后另一块是否继续工作
- 如何避免形成复杂双主架构
- 谁拥有最终执行器使能权限
- RK3588是否负责最终状态仲裁

- 硬件急停是否绕过所有软件仲裁

默认建议：

- 两块STM32只交换心跳、安全状态和关键系统状态
- 不互相发送普通执行器控制命令
- 不自动接管对方执行器
- 每个执行器拥有唯一控制所有者
- RK3588负责正常状态汇总
- STM32负责本地安全保护
- 硬件急停拥有最高优先级

请给出推荐方案及通信数据定义。

十九、参数与配置管理

请设计三级参数体系：

1. 出厂参数
2. 机器人型号参数
3. 运行时可调参数

参数至少包含：

- 参数ID
- 名称
- 类型
- 默认值
- 最小值
- 最大值
- 单位
- 权限
- 所属节点
- 生效方式
- 是否需要重启
- 是否允许热更新
- 版本
- CRC

请说明：

- 参数存储位置
- RK和STM32各自保存哪些参数
- 参数单一事实源
- 参数同步机制
- 两块STM32参数一致性
- 参数修改权限
- 参数修改认证
- 参数掉电保护
- 双区存储

- 原子更新
- 参数回滚
- 参数版本升级
- 参数迁移
- 工厂恢复
- 参数写入寿命
- Flash磨损均衡

请设计参数更新事务：

- 开始更新
- 参数校验
- 暂存
- 完整性检查
- 提交
- 生效
- 回滚

二十、固件升级架构

请设计以下升级流程：

- RK3588系统升级
- RK3588应用升级
- MCU-A应用升级
- MCU-B应用升级
- MCU-A Bootloader升级
- MCU-B Bootloader升级
- USB升级
- SPI在线升级
- USART救援升级

升级机制至少包含：

- 升级包格式
- 目标节点
- 硬件型号
- 固件角色
- 版本号
- 最低兼容版本
- 文件长度
- 哈希
- 数字签名
- CRC
- 分片
- 断点续传
- 超时
- 重传
- 双分区

- 回滚
- 防降级
- 升级日志
- 升级状态上报

请重点分析：

1. 两块STM32的升级顺序。
2. 升级MCU-B时MCU-A能否继续运行。
3. 升级MCU-A时机器人必须处于何种状态。
4. 两块STM32固件版本不兼容时是否允许启动。
5. 升级失败后如何进入旧版本。
6. Bootloader损坏后如何救援。
7. USART救援升级是否需要物理操作授权。
8. 如何避免未经授权的固件运行。
9. 如何保证升级期间执行器安全。
10. 是否需要A/B镜像。

请给出完整升级状态机。

二十一、故障诊断与日志

建立统一故障码体系。

每个故障至少包含：

- 故障模块
- 节点ID
- 通道ID
- 设备ID
- 故障等级
- 故障码
- 首次发生时间
- 最近发生时间
- 发生次数
- 当前状态
- 恢复状态
- 是否锁存
- 自动恢复条件
- 人工复位要求
- 上下文快照
- 固件版本
- 通信统计

故障等级建议：

- INFO
- WARNING
- DEGRADED
- ERROR

- CRITICAL

请设计：

- STM32本地故障环形记录
- RK3588集中故障管理
- 高频故障合并
- 故障限流
- 故障去抖
- 故障锁存
- 故障恢复
- 掉电保存
- 黑匣子数据
- 关键变量快照
- 重启原因记录
- 看门狗复位记录
- Bus Off记录
- 设备掉线记录
- 参数修改记录
- 升级记录

请说明：

1. 哪些日志保存在STM32？
2. 哪些日志上传RK？
3. STM32离线后如何保留故障？
4. 高频故障如何防止刷爆Flash？
5. 日志是否允许阻塞实时任务？
6. 如何实现日志等级和限流？
7. 如何导出黑匣子数据？

二十二、看门狗和系统健康监控

请设计：

- STM32独立看门狗
- FreeRTOS任务健康监控
- RK进程看门狗
- RK系统看门狗
- MCU心跳监控
- 外部硬件看门狗

要求：

1. 每个关键任务必须独立上报活跃状态。
2. WatchdogTask只能在所有关键任务满足条件后喂狗。
3. 不能由单个任务直接无条件喂狗。
4. 任务活跃不只检查“执行过”，还要检查：

5. 周期是否正常
6. 执行时间是否超限
7. 状态机是否推进
8. 数据是否更新
9. 看门狗复位原因必须记录。
10. 多次连续复位后应进入安全模式或救援模式。

请给出任务健康表和喂狗伪代码。

二十三、时间同步

请设计RK3588与两块STM32的时间同步机制。

需要考虑：

- RK系统时间
- 单调时钟
- STM32本地Tick
- 硬件定时器
- 协议时间戳
- 时间漂移
- 往返时延
- SPI轮询延迟
- 两块STM32之间的时间一致性
- 日志对时
- 传感器时间戳
- 控制命令有效期

请比较：

- 简单周期校时
- NTP式往返计算
- PTP思想的简化实现
- GPIO同步脉冲
- PPS信号

请给出推荐精度目标和实现方法。

二十四、性能预算

请提供性能预算表和计算方法。

至少包括：

- SPI时钟
- SPI有效吞吐量

- SPI利用率
- 单次事务长度
- SPI轮询周期
- RK到STM32控制延迟
- STM32到RK状态延迟
- 控制抖动
- 控制命令频率
- 状态反馈频率
- CAN FD总线负载
- CAN FD帧周期
- RS485轮询周期
- RS485单设备响应时间
- STM32 CPU占用率
- 中断占用率
- FreeRTOS上下文切换开销
- RAM预算
- Flash预算
- 任务栈预算
- 消息队列预算
- DMA缓冲区预算
- 日志带宽
- 升级带宽

无法给出准确数值时：

1. 给出计算公式。
2. 给出示例参数。
3. 标明假设。
4. 给出推荐余量。
5. 不要凭空给出最终确定值。

建议目标：

- STM32平均CPU占用率保留足够余量
- CAN FD总线负载保留峰值余量
- RS485轮询周期考虑最坏设备响应
- SPI链路预留升级和日志带宽
- RAM和任务栈保留安全余量

二十五、代码仓库组织

请给出可直接使用的代码目录结构。

建议参考：

```
robot_controller/
├── rk3588/
│   ├── drivers/
│   └── transport/
```

```

|   ├── protocol/
|   ├── mcu_gateway/
|   ├── device_manager/
|   ├── motion_manager/
|   ├── diagnostics/
|   ├── parameter_service/
|   ├── update_service/
|   ├── health_manager/
|   ├── logger/
|   ├── ros2_adapter/
|   └── applications/
|
|   ├── mcu_common/
|   |   ├── bsp_interface/
|   |   ├── drivers/
|   |   ├── osal/
|   |   ├── transport/
|   |   ├── protocol/
|   |   ├── canfd/
|   |   ├── rs485/
|   |   ├── device_framework/
|   |   ├── diagnostics/
|   |   ├── parameters/
|   |   ├── safety/
|   |   ├── logger/
|   |   ├── watchdog/
|   |   └── bootloader/
|   |
|   |   ├── mcu_a/
|   |   |   ├── board/
|   |   |   ├── config/
|   |   |   ├── devices/
|   |   |   ├── control/
|   |   |   ├── safety_policy/
|   |   |   └── application/
|   |   |
|   |   |   ├── mcu_b/
|   |   |   |   ├── board/
|   |   |   |   ├── config/
|   |   |   |   ├── devices/
|   |   |   |   ├── control/
|   |   |   |   ├── safety_policy/
|   |   |   |   └── application/
|   |   |
|   |   |   ├── protocol/
|   |   |   ├── idl/
|   |   |   ├── generated/
|   |   |   ├── bootloader/
|   |   |   ├── tools/
|   |   |   ├── tests/
|   |   |   └── unit/

```

```
|   |—— integration/
|   |—— protocol/
|   |—— mock/
|   |—— fault_injection/
|—— simulation/
|—— hil/
|—— docs/
|—— ci/
```

请说明：

- 哪些代码由MCU-A和MCU-B共享
- 哪些代码必须独立
- Board层如何隔离引脚差异
- 产品型号如何配置
- 是否采用CMake
- 是否采用Kconfig
- 是否采用YAML配置生成
- STM32CubeMX生成代码如何管理
- 自动生成代码如何隔离
- 如何避免修改CubeMX生成文件
- 如何组织Mock驱动
- 如何在PC上测试协议和状态机
- 如何避免公共模块依赖具体业务

禁止通过大量条件编译维护角色差异，例如：

```
#ifdef MCU_A
...
#else
...
#endif
```

优先采用：

- 配置结构
- 设备描述表
- 函数指针
- 接口注入
- 编译目标配置
- Kconfig
- 自动代码生成

二十六、核心接口设计

请给出关键接口定义示例。

至少包含：

Transport接口

```
typedef struct
{
    int (*open)(void *context);
    int (*close)(void *context);
    int (*send)(void *context, const uint8_t *data, uint32_t length);
    int (*receive)(void *context, uint8_t *data, uint32_t capacity);
    int (*get_status)(void *context, void *status);
} TransportInterface;
```

CAN FD接口

```
typedef struct
{
    int (*init)(CanFdChannel *channel);
    int (*send)(CanFdChannel *channel, const CanFdFrame *frame);
    int (*receive)(CanFdChannel *channel, CanFdFrame *frame);
    int (*recover_bus_off)(CanFdChannel *channel);
    int (*get_statistics)(CanFdChannel *channel,
        CanFdStatistics *statistics);
} CanFdInterface;
```

RS485接口

```
typedef struct
{
    int (*submit_transaction)(Rs485Channel *channel,
        const Rs485Transaction *transaction);
    int (*cancel_transaction)(Rs485Channel *channel,
        uint32_t transaction_id);
    int (*poll)(Rs485Channel *channel);
    int (*get_statistics)(Rs485Channel *channel,
        Rs485Statistics *statistics);
} Rs485Interface;
```

设备接口

```
typedef struct
{
    int (*init)(void *device);
    int (*start)(void *device);
    int (*stop)(void *device);
}
```

```
int (*process)(void *device, uint32_t now_ms);
int (*set_command)(void *device, const void *command);
int (*get_state)(void *device, void *state);
int (*get_fault)(void *device, void *fault);
} DeviceInterface;
```

请进一步完善这些接口，并说明：

- 返回值规范
- 线程安全要求
- 所有权
- 生命周期
- 超时
- 回调
- 事件通知
- 内存管理
- 错误传播

二十七、测试方案

请设计完整测试体系。

至少包括：

1. 纯算法单元测试
2. 协议编解码测试
3. 协议兼容性测试
4. CRC测试
5. 序列号和去重测试
6. 驱动Mock测试
7. 设备状态机测试
8. FreeRTOS任务测试
9. CAN FD虚拟节点测试
10. CANopen节点测试
11. RS485虚拟设备测试
12. Modbus RTU测试
13. SPI压力测试
14. USB断连重连测试
15. USART救援升级测试
16. RK与STM32通信压力测试
17. 长时间稳定性测试
18. 故障注入测试
19. 通信丢帧测试
20. 通信乱序测试
21. 重复帧测试
22. CRC错误测试
23. SPI事务中断测试
24. CAN Bus Off测试

25. CAN总线拥塞测试
26. RS485设备不响应测试
27. RS485设备持续占线测试
28. STM32异常复位测试
29. RK3588进程崩溃测试
30. RK3588系统重启测试
31. 参数写入断电测试
32. 升级断电测试
33. 升级失败回滚测试
34. 看门狗测试
35. 急停测试
36. 硬件在环测试

请给出：

- 测试层级
- 测试工具
- 测试输入
- 预期结果
- 通过标准
- 开发阶段准入条件

开发阶段至少分为：

- 架构验证阶段
- 驱动开发阶段
- 协议联调阶段
- 设备接入阶段
- 整机联调阶段
- 产品验证阶段
- 量产阶段

二十八、CI/CD和质量要求

请设计：

- RK代码构建
- STM32代码构建
- 多目标构建
- 自动代码生成检查
- 静态分析
- 格式检查
- 单元测试
- 协议兼容检查
- 固件大小检查
- RAM使用检查
- 栈使用检查
- MISRA检查
- Coverity或Cppcheck

- clang-tidy
- 代码覆盖率
- 自动发布
- 固件签名
- 版本号生成
- 构建产物归档

请说明：

- 每次提交必须通过哪些检查
- 合并请求必须满足哪些条件
- 发布版本如何生成
- MCU-A和MCU-B如何保持协议兼容
- 自动生成文件变化如何审查

二十九、架构风险审查

在完成方案后，请主动进行一次架构自审。

至少检查：

1. RK3588和STM32是否存在职责重叠。
2. MCU-A和MCU-B是否可能同时控制同一执行器。
3. SPI、USB和USART是否存在多主控制风险。
4. 链路切换是否可能导致旧命令重复执行。
5. Linux进程崩溃是否可能导致保持最后控制量。
6. 控制命令是否具有序列号、时间戳、有效期和会话ID。
7. 协议是否能处理任一端复位。
8. 是否错误地对实时控制命令无限重传。
9. FreeRTOS任务是否存在优先级反转。
10. 是否可能发生高优先级任务饿死低优先级任务。
11. 是否在中断中执行耗时操作。
12. CAN FD高负载时是否可能丢失关键控制帧。
13. Bus Off恢复是否安全。
14. RS485单设备超时是否阻塞全部设备。
15. 是否存在全局变量导致多通道干扰。
16. 参数更新是否具有原子性。
17. 固件升级失败后是否能恢复。
18. 两块STM32版本不匹配时是否允许运行。
19. 故障码是否能定位到节点、通道和设备。
20. 看门狗是否可能被错误喂养。
21. 硬件急停和软件急停是否被混淆。
22. 功能安全能力是否被夸大。
23. 协议、任务、内存和带宽是否有量化依据。
24. 架构是否存在不必要的复杂度。
25. 测试方案是否覆盖关键故障。

请输出：

- 严重问题
 - 高风险问题
 - 一般问题
 - 触发条件
 - 可能后果
 - 修改建议
 - 是否可以进入详细设计阶段
-

三十、最终输出格式

请严格按照以下顺序输出完整方案：

1. 需求理解
2. 已知硬件条件
3. 关键假设
4. 架构约束
5. 硬实时、软实时和非实时功能划分
6. 双STM32职责方案对比
7. 推荐的双STM32职责划分
8. 推荐硬件通信拓扑
9. SPI、USB和USART职责矩阵
10. 系统总体架构图，使用Mermaid
11. RK3588软件分层图
12. STM32软件分层图
13. MCU-A功能模块
14. MCU-B功能模块
15. RK侧多MCU实例设计
16. STM32多通道对象模型
17. FreeRTOS任务设计表
18. CAN FD模块设计
19. RS485模块设计
20. RK与STM32统一协议
21. SPI通信机制
22. 关键通信时序图，使用Mermaid
23. 数据字典与代码生成
24. 系统安全状态机，使用Mermaid
25. 双STM32协同机制
26. 故障处理矩阵
27. 参数管理方案
28. 固件升级方案
29. 故障诊断与日志方案
30. 看门狗和健康监控
31. 时间同步方案
32. 性能预算表
33. RAM、Flash和任务栈预算
34. 代码仓库目录树
35. 核心接口定义

36. 关键模块伪代码
 37. 测试方案
 38. CI/CD方案
 39. 分阶段开发计划
 40. 架构风险审查
 41. 需要硬件团队确认的事项
 42. 需要产品团队确认的事项
 43. 是否可以进入详细设计阶段的结论
-

三十一、回答要求

1. 使用中文回答。
2. 不要只给概念，要给工程实现方法。
3. 不要因为部分参数未确定而停止分析。
4. 对未知项先提出合理假设，再继续设计。
5. 所有关键决策必须说明理由和代价。
6. 架构图和状态机尽量使用Mermaid。
7. 任务、资源、协议和性能尽量使用表格。
8. 代码示例使用C或C++。
9. STM32侧代码应适合裸机与FreeRTOS混合开发习惯。
10. RK3588侧优先使用现代C++，但避免过度复杂的模板设计。
11. 明确指出哪些内容需要进一步通过硬件原理图确认。
12. 特别核查STM32G474具体封装的：
 - FDCAN实例
 - UART实例
 - SPI实例
 - USB实例
 - DMA资源
 - 中断资源
 - 引脚复用
13. 特别核查2路CAN FD、4路RS485、SPI从机、USB、USART和调试接口能否在同一封装下同时使用。
14. 不要默认所有HAL外设都能同时使用，必须考虑引脚和DMA冲突。
15. 对无法确认的硬件能力明确标记为“需要原理图或具体型号确认”。
16. 最终方案应能直接指导后续详细设计和代码仓库搭建。